خدمتكم عبادة نتقرب بها الى الله

2021

سلايدات

م.سماح

مهارت الحاسوب 2



سنافر البوليتكنك 2021

Computer Skills II
C++
Summer Semester 2019/2020

Prepared by : MEng. Samah A. Massadeh

Introduction

What is computer?

- **Computer**
  - A device capable of performing computations and making logical decisions
  - A machine that manipulates data according to a list of instructions.
  - A programmable device that can store, retrieve, and process data.

- **Computer programs**
  - Sets of instructions that control a computer's processing of data

- **Hardware**
  - Physical part of the computer
  - Various devices comprising a computer
    - Examples: keyboard, screen, mouse, disks, memory, CD-ROM, and processing units

- **Software**
  - A collection of computer programs, procedures and documentation that perform some tasks on a computer system
  - Programs that run a computer

Computer languages

- Computer languages are divided into three types.
  - **Machine languages**
    - Machine language is machine dependent.
    - Strings of numbers giving machine specific instructions
    - Example:

```
+1300042774
+1400593419
+1200274027
```

  - **Assembly languages**
    - English-like abbreviations representing elementary computer operations (translated via assemblers)
    - Example:

```
LOAD BASEPAY
ADD OVERPAY
STORE GROSSPAY
```

Translator programs called **assembler** were developed to convert assembly language programs to machine language programs at computer speed.

Computer languages

- **High-level languages**
  - Similar to everyday English, use mathematical notations (translated via compilers)
  - Example:

```
grossPay = basePay + overTimePay
```

- C, C++ are the most widely used high level languages. Some other examples are
- FORTRAN (formula translator)

  Used in scientific and engineering applications
- COBOL (common business oriented language)

  Used to manipulate large amounts of data
- Pascal

  Used to teach structured programming
- Translator programs called **Compilers** converts high-level language programs into machine language

C++ Programming

Simple program to print a line of text.

```
1  // A first program in C++
2  #in
3
4  int
5  {
6     cout << "
7
8     return 0;      // indicate that pro
successfully
9  }
```

Comments

etween **/*** and ***/** or following a **//**.

rogram readability and do not cause the to perform any action.

return is a way to exit a function from a function.

return 0, in this case, means that the program terminated normally. It is one of the several means used to exit a function

preprocessor directive

reprocessor.

are preprocessor directives.

eam> tells the preprocessor

The left brace **,{,** line 5 must begin the body of function and the corresponding right brace **,}**, line 9 must end the body of each function.

one or more functions, one of which must be **main**

Prints the *string* of characters contained between the quotation marks.

The entire line, including **cout**, the **<<** *operator*, the *string* **"Welcome to C++!\n"** and the *semicolon* (**;**), is called a *statement*.
All statements must end with a semicolon.

C++ Programming

- **cout**
  - Standard output stream object
  - “Connected” to the screen

- **<<**
  - Stream insertion operator
  - Value to the right of the operator (right operand) inserted into output stream (which is connected to the screen)
  - `cout << “Welcome to C++!\n”;`

C++ Programming

```
1  // an example to observe using statement
2  // program to display greeting
3  #include <iostream.h>
4
5  int main()
6  {
7     cout << "Hello world\n";
8
9     return 0;      // indicate that program ended successfully
10 }
```

Escape Character

- Indicates that a “special” character is to be output

Escape Sequence	Description
\n	Newline. Position the screen cursor to the beginning of the next line.
\t	Horizontal tab. Move the screen cursor to the next tab stop.
\r	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
\a	Alert. Sound the system bell.
\\	Backslash. Used to print a backslash character.
\"	Double quote. Used to print a double quote character.
\b	backspace

C++ Programming

- There are multiple ways to print text. Following are some more examples.

```
1  //observing the use of \n
2  // Printing a line with multiple statements
3  #include <iostream.h>
4
5  int main()
6  {
7     cout << "Welcome ";
8     cout << "to C++!\n";
9
10    return 0;   // indicate that program ended successfully
11 }
```

The output would be as bellow

Welcome to C++!

C++ Programming

```
1  // printing multiple lines with a single statement
2  // Printing multiple lines with a single statement
3  #include <iostream.h>
4
5  int main()
6  {
7     cout << "Welcome\nto\n\nC++!\n";
8
9     return 0;   // indicate that program ended successfully
10 }
```

```
Welcome
To

C++!
```

Fundamentals of C++

Fundamental C++ Objects

- C++ has a large number of fundamental or built-in object types
- The fundamental object types fall into one of three categories
  - Integer objects
  - Floating-point objects
  - Character objects

Integer object type

- The basic integer object type is **`int`**
  - The size of an **`int`** depends on the machine and the compiler
    - On PCs it is normally 16 or 32 bits
- Other integers object types
  - **`short`**: typically uses less bits (often 2 bytes)
  - **`long`**: typically uses more bits (often 4 bytes)
- Different types allow programmers to use resources more efficiently
- Standard arithmetic and relational operations are available for these types

- Integer constants are positive or negative whole numbers

Decimal Constants

- Examples
  - 97
  - 40000
  - 50000
  - -65
  - 23a (illegal)

Floating-point object

- Floating-point object types represent real numbers
  - Integer part
  - Fractional part
- The number 108.1517 breaks down into the following parts
  - 108 - integer part
  - 1517 - fractional part
- C++ provides three floating-point object types
  - **`float`**
    - (often 4 bytes) Declares floating point numbers with up to 7 significant digits
  - **`double`**
  - **`long double`**
    - `(often 10 bytes)` Declares floating point numbers with up to 19 significant digits.

Character object type

- **Char** is for specifying character data
- char variable may hold only a single lowercase letter, a single upper case letter, a single digit, or a single special character like a $, 7, *, etc.
- case sensitive, i.e. a and A are not same.
- ASCII is the dominant encoding scheme
  - Examples
    - `' '` encoded as 32 `'+'` encoded as 43
    - `'A'` encoded as 65 `'Z'` encoded as 90
    - `'a'` encoded as 97 `'z'` encoded as 122

Character object type

- **Explicit (literal) characters within single quotes**
  - `'a','D','*'`

Special characters - delineated by a backslash \

- Two character sequences (escape codes)
- Some important special escape codes
  - `\t` denotes a tab
  - `\n` denotes a new line
  - `\\` denotes a backslash
  - `\'` denotes a single quote
  - `\"` denotes a double quote
  - `\r` denotes carriage return
  - `\b` denotes backspace
- `'\t'` is the explicit tab character, `'\n'` is the explicit new line character, and so on

Memory Concepts

- **Variable**
  - Variables are names of memory locations
  - Correspond to actual locations in computer's memory
  - Every variable has name, type, size and value
  - When new value placed into variable, overwrites previous value

```
int integer1;
integer1=45;

int integer2;
integer2=72;
int sum;

sum = integer1 + integer2;
```







Names (naming entities)

- Used to denote program values or components
- A valid name is a sequence of
  - Letters (upper and lowercase)
  - A name cannot start with a digit
- Names are case sensitive
  - MyObject is a different name than MYOBJECT
- There are two kinds of names
  - Keywords
  - Identifiers

Keywords

- Keywords are words reserved as part of the language
  - `int`, `return`, `float`, `double`
- They cannot be used by the programmer to name things
- They consist of lowercase letters only
- They have special meaning to the compiler

C++ key words

auto	break	case	char	const
continue	default	do	double	else
enum	extern	float	for	goto
if	int	long	register	return
short	signed	sizeof	static	struct
switch	typedef	union	unsigned	void
volatile	while			
C++ only keywords				
asm	bool	catch	class	const_cast
delete	dynamic_cast	explicit	false	friend
inline	mutable	namespace	new	operator
private	protected	public	reinterpret_cast	
static_cast	template	this	throw	true
try	typeid	typename	using	virtual
wchar_t				

Identifiers

- Identifiers are used to name entities in c++
  - It consists of letters, digits or underscore
  - Starts with a letter or underscore
  - Can not start with a digit

- Examples
  - `Grade`
  - `Temperature`
  - `CameraAngle`
  - `IntegerValue`

Definitions/declaration

Location in memory where a value can be stored for program use

- All objects (or variable) that are used in a program must be defined (declared)
- An object definition specifies
  - Type
  - Identifier
- General definition form

Known type

List of one or more identifiers

```
Type Id, Id, ..., Id;
```

Examples

```
char  Response;
int   MinElement;
float Score;
float Temperature;
int  i;
int n;
char c;
float x;
```

(Value of an object is whatever is in its assigned memory location)

Type compatibilities

- Rule is to store the values in variables of the same type
  - This is a type mismatch:

```
int int_variable;
int_variable = 2.99;
```

  - If your compiler allows this, int variable will most likely contain the value 2, not 2.99

Stream extraction and assignment operator

- **>>** (stream extraction operator)
  - When used with **cin**, waits for the user to input a value and stores the value in the variable to the right of the operator
  - The user types a value, then presses the *Enter* (Return) key to send the data to the computer
  - Example:

```
int  myVariable;
cin >> myVariable;
```

    - Waits for user input, then stores input in **myVariable**
- **=** (assignment operator)
  - Assigns value to a variable
  - Binary operator (has two operands)
  - Example:

```
sum = variable1 + variable2;
```

A simple program to add two numbers

```
1  //example
2  // program to add two numbers
3  #include <iostream.h>
4
5  int main()
6  {
7     int integer1, integer2, s
8
9     cout << "Enter first inte
10    cin >> integer1;
11    cout << "Enter seco
12    cin >> integer2;
13    sum = integer1 + in
14    cout << "Sum is " << sum << endl; // print
15
16    re
17 }
```

Use stream extraction operator with standard input stream to obtain user input.

Calculations can be performed in output statements: alternative for lines 13 and 14:

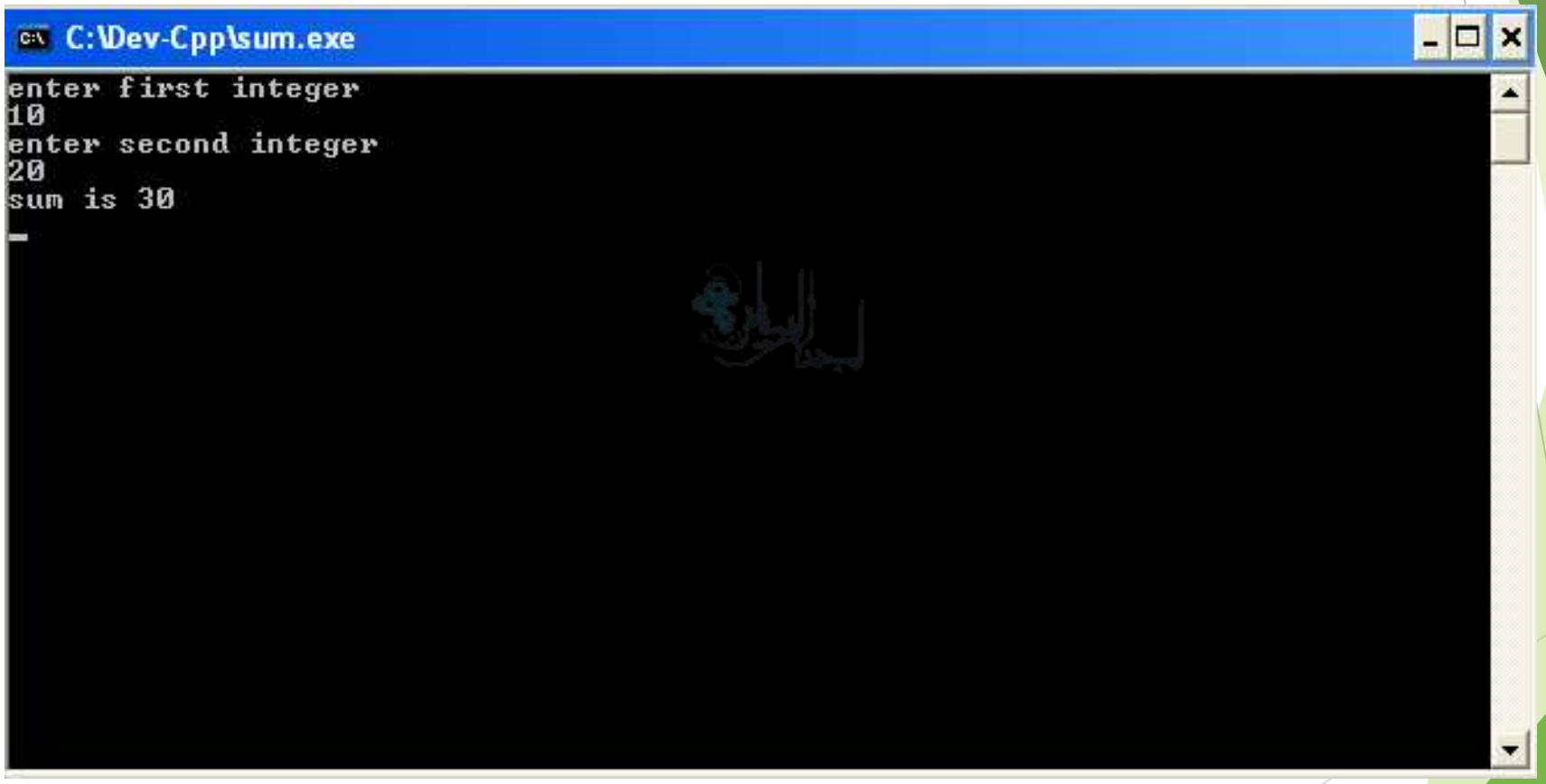
```
cout << "Sum is " << integer1 + integer2 << std::endl;
```

•Note that data entered from the keyboard must be compatible with the data type of the

•Variables can be output using **cout << variableName**.
- •Generl form is cout<<expression;
- •An expression is any c++ expression(str
formula or function call)
- •Cout is an o stream object
- •streams output to standard output
- •uses the << (output) operator

Concatenating, chaining or cascading stream insertion operations.

Output of program



program to find the area of rectangle

```
// program to find the area of a rectangle

#include <iostream>
using namespace std;

int main()
{
    float length, width, area;                  // declearation
    cout<< " enter length of rectagle \t";      // prompt to enter lengt
    cin>>length;                                // reading length

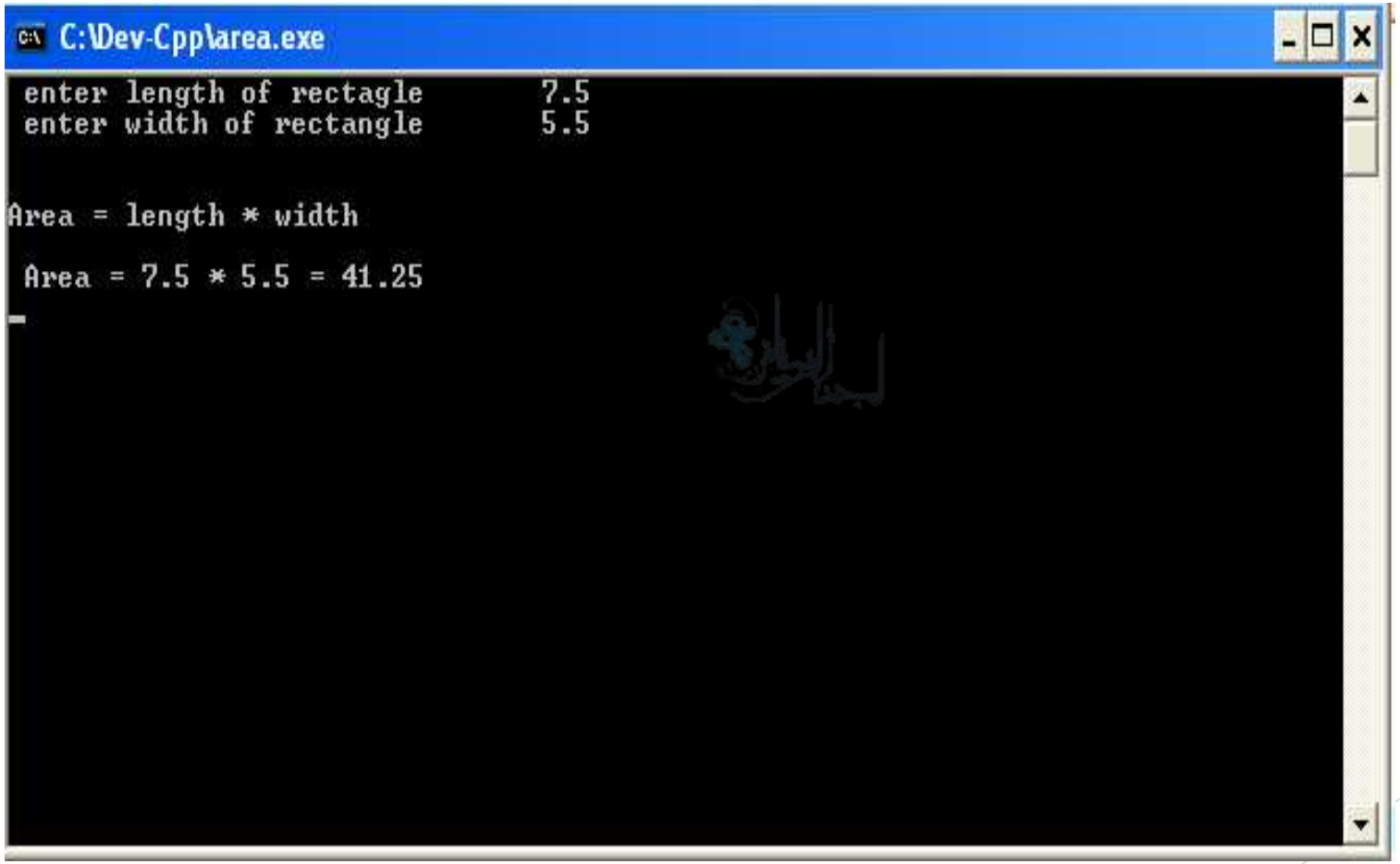
    cout<< " enter width of rectangle \t";      // prompt to enter width
    cin>>width;                                 //reading width

    area = length * width;
                                                //printing result
    cout<<" \n\nArea = length * width \n\n";
    cout<<" Area = "<<length<<" * "<<width<<" = "<<area<<endl;

    //indicates that program ended successfully
    return 0;
```

Tells the compiler to use names in iostream in a “standard” way

output


C:\Dev-Cpp\area.exe
enter length of rectagle 7.5
enter width of rectangle 5.5
Area = length * width
Area = 7.5 * 5.5 = 41.25

Arithmetic and Logical operators

Arithmetic Operators

- Arithmetic operators are listed in following table

C++ operation	Arithmetic operator	Algebraic expression	C++ expression
Addition	+	$f + 7$	`f + 7`
Subtraction	-	$p - c$	`p - c`
Multiplication	*	bm	`b * m`
Division	/	x / y	`x / y`
Modulus	%	$r \bmod s$	`r % s`

- Modulus operator returns the remainder of integer division

  `7 % 5` evaluates to 2

- Integer division truncates remainder

  `7 / 5` evaluates to 1

Results of Arithmetic operators

- Arithmetic operators can be used with any numeric type.

- An **operand** is a number or variable used by the operator e.g.
  - integer1 + integer2
    - + is operator
    - integer1 and integer2 are operands

- Result of an operator depends on the types of operands
  - If both operands are int, the result is int
  - If one or both operands are double, the result is double

Integer Division



Examples comparing mathematical and C++ expressions

Mathematical Formula	C++ Expression
$b^2 - 4ac$	b*b – 4*a*c
$x(y + z)$	x*(y + z)
$\frac{1}{x^2 + x + 3}$	1/(x*x + x + 3)
$\frac{a + b}{c - d}$	(a + b)/(c – d)

Operator precedence

- Some arithmetic operators act before others (e.g., multiplication before addition)
  - Be sure to use parenthesis when needed
- Example: Find the average of three variables a, b and c
  - Do not use: `a + b + c / 3 (incorrect)`
  - Use: `(a + b + c ) / 3 (correct)`

Rules of operator precedence

Operator(s)	Operation(s)	Order of evaluation (precedence)
`()`	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses "on the same level" (i.e., not nested), they are evaluated left to right.
`*`, `/`, or `%`	Multiplication Division Modulus	Evaluated second. If there are several, they are evaluated left to right.
`+` or `-`	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

Operator Precedence

An example to understand operator precedence.

```
20 -  4 / 5  * 2   +   3 * 5  % 4
```

```
      (4 / 5)
        ((4 / 5) * 2)
        ((4 / 5) * 2)       (3 * 5)
        ((4 / 5) * 2)     ((3 * 5) % 4)
   (20 -((4 / 5) * 2))    ((3 * 5) % 4)
   (20 -((4 / 5) * 2)) + ((3 * 5) % 4)
```

Assignment operators

- = is the assignment operator
- Used to assign a value to a variable
- An assignment statement changes the value of a variable
- General Form:
  identifier = expression;

- The single variable to be changed is always on the left of the assignment operator '='
- On the right of the assignment operator can be
  - Constants
    - For example age = 21;
  - Variables
    - For example my_cost = your_cost;
  - Expressions
    - For example circumference = diameter * 3.14159;

Assignment operators

- The '=' operator in C++ is not an equal sign
  - The following statement cannot be necessarily true in algebra

```
number_of_bars =  number_of_bars  +  3;
```

  - In C++ it means the **new** value of number_of_bars is the **previous** value of number_of_bars plus 3

Assignment expression abbreviations

- Program can be written and compiled a bit faster by the use of abbreviated assignment operators
- C++ provides several assignment operators for abbreviating assignment expressions.
- **Addition assignment operator**

```
c = c + 3;
```
abbreviated to
```
c += 3;
```

- **Statements of the form**

```
variable = variable operator expression;
```
can be rewritten as
```
variable operator= expression;
```

- **Other assignment operators**

```
d -= 4      (d = d - 4)
e *= 5      (e = e * 5)
f /= 3      (f = f / 3)
g %= 9      (g = g % 9)
```

Arithmetic assignment operators

Assignment operator	Sample expression	Explanation	Assigns
Assume: `int c = 3, d = 5, e = 4, f = 6, g = 12;`			
`+=`	`c += 7`	`c = c + 7`	10 to `c`
`-=`	`d -= 4`	`d = d - 4`	1 to `d`
`*=`	`e *= 5`	`e = e * 5`	20 to `e`
`/=`	`f /= 3`	`f = f / 3`	2 to `f`
`%=`	`g %= 9`	`g = g % 9`	3 to `g`

Increment and Decrement Operators

- Increment and decrement operators are unary operators as they require only one operand.
- **++ unary increment operator**
  - Adds 1 to the value of a variable

    x ++;

    is equivalent to x = x + 1;
  - **Pre-increment**
    - When the operator is used before the variable (++c)
    - Variable is changed, then the expression it is in is evaluated
  - **Post-increment**
    - When the operator is used after the variable (c++)
    - Expression the variable is in executes, then the variable is changed.

Increment and Decrement Operators

- **-- -- unary decrement operator**
  - Subtracts 1 from the value of a variable

    x --;

    is equivalent to x = x – 1;
  - **Pre-decrement**
    - When the operator is used before the variable (`--c`)
    - Variable is changed, then the expression it is in is evaluated.
  - **Post-decrement**
    - When the operator is used after the variable (`c--`)
    - Expression the variable is in executes, then the variable is changed.

Increment and Decrement Operators

- **Example**
- If `c = 5`, then
  - `cout << ++c;`
    - `c` is changed to `6`, then printed out
  - `cout << c++;`
    - Prints out `5` (`cout` is executed before the increment)
    - `c` then becomes `6`

- **When variable not in expression**
  - Preincrementing and postincrementing have same effect

```
++c;
cout << c;
```

and

```
c++;
cout << c;
```

are the same

Summarizing increment and decrement operators in a table

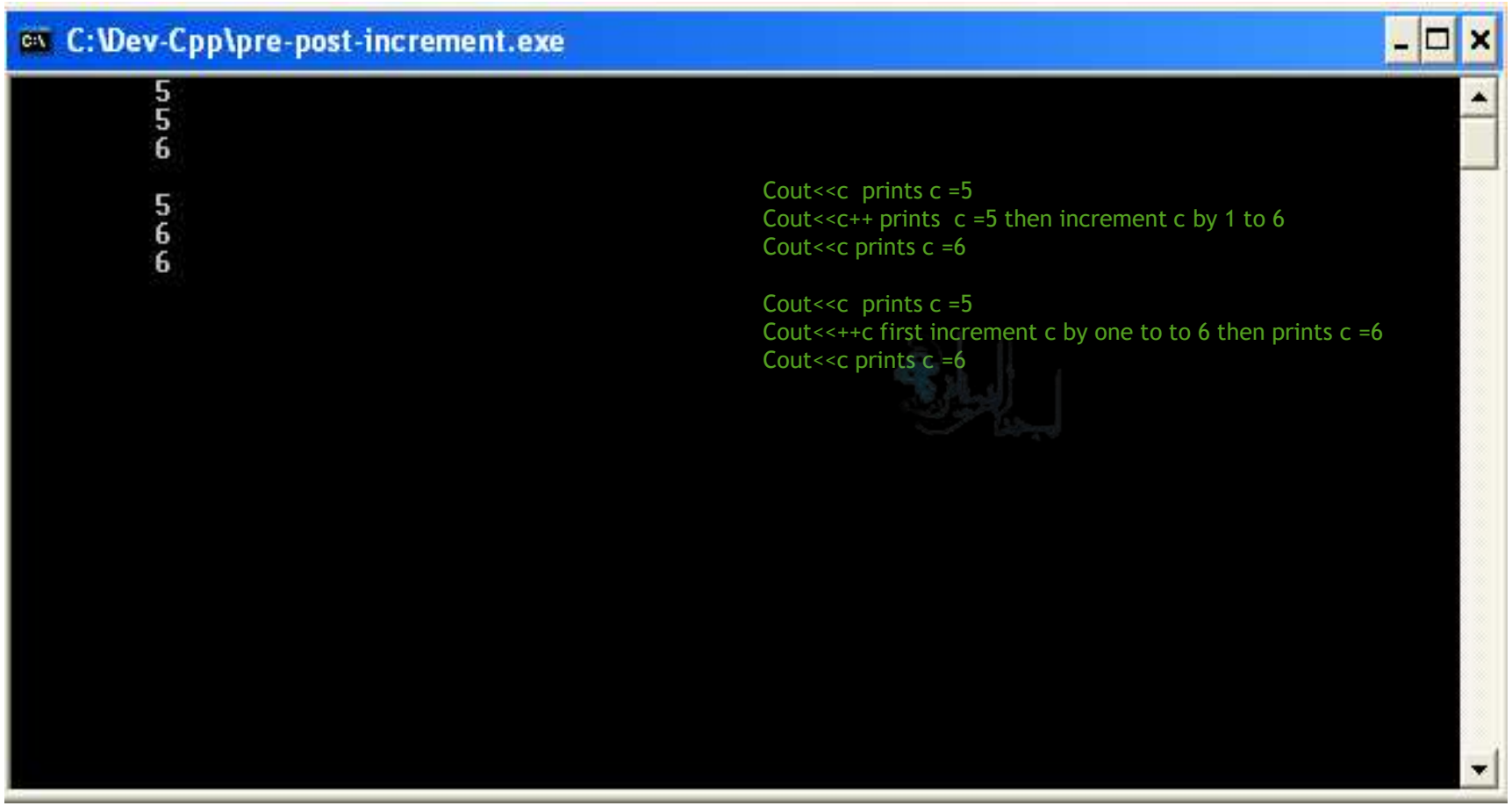
Operator	Called	Sample expression	Explanation
`++`	preincrement	`++a`	Increment **a** by 1, then use the new value of **a** in the expression in which **a** resides.
`++`	postincrement	`a++`	Use the current value of **a** in the expression in which **a** resides, then increment **a** by 1.
`--`	predecrement	`--b`	Decrement **b** by 1, then use the new value of **b** in the expression in which **b** resides.
`--`	postdecrement	`b--`	Use the current value of **b** in the expression in which **b** resides, then decrement **b** by 1.

The associativity of these unary operators is from right to left

An example to understand the effect of pre-increment and post-increment

```
1   // example
2   // Pre incrementing and post incrementing.
3   #include <iostream.h>
4
5
8   // function main begins program execution
9   int main()
10  {
11    int c;                  // declare variable
12  13      // demonstrate pos tincrement
14    c = 5;                  // assign 5 to c
15    cout << c << endl;        // print 5
16    cout << c++ << endl;      // print 5 then post increment
17    cout << c << endl << endl; // print 6
18
19    // demonstrate pre increment
20    c = 5;                  // assign 5 to c
21    cout << c << endl;        // print 5
22    cout << ++c << endl;      // pre increment then print 6
23        cout << c << endl;        // print 6
24
25    return 0;   // indicate successful termination
26
27  } // end function main
```

output


C:\Dev-Cpp\pre-post-increment.exe
5
5
6
5
6
6
Cout<<c prints c =5
Cout<<c++ prints c =5 then increment c by 1 to 6
Cout<<c prints c =6
Cout<<c prints c =5
Cout<<++c first increment c by one to to 6 then prints c =6
Cout<<c prints c =6

Precedence of the operators encountered so far

Operators	Associativity	Type
`()`	left to right	parentheses
`* / %`	left to right	multiplicative
`+ -`	left to right	additive
`< <= > >=`	left to right	relational
`== !=`	left to right	equality
`= += -= *= /= %=`	right to left	assignment

Logical Operators

- C++ logical operators are

&& (Logical AND)

|| (Logical OR)

! (Logical NOT , logical negation)

Logical Operators

- **&& (Logical AND)**
  - All of the conditions must be true for the compound condition to be true
  - **Syntax** (*Condition_1*) && (*Condition_2*)
  - True if both expressions are true
  - **Example**
    - **if ((2 < x) && (x < 7))**
  - True only if x is between 2 and 7
  - Inside parentheses are optional but enhance meaning

Truth table for && (AND) operator

Value of *condition1*	Value of *condition2*	Value of *condition1 && condition2*
true	true	true
true	false	false
false	true	false
false	false	false

Logical Operators

- Be careful translating inequalities to C++
- if **x < y < z** translates as

if ((x < y) && (y < z))

NOT

if (x < y < z)

Logical Operators

- **|| (Logical OR)**
  - only one of the conditions must be true for the compound condition to be true
  - True if either or both expressions are true
  - **Syntax** (Condition_1) || (Condition_2)
  - Example
    - **if ((x = = 1) | | (x = = y))**
  - True if x contains 1
  - True if x contains the same value as y
  - True if both comparisons are true

Truth table for the || (OR) operator

Value of *condition1*	Value of *condition2*	Value of *condition1* || *condition2*
true	true	true
true	false	true
false	true	true
false	false	false

Logical Operators

- **! (Logical NOT , logical negation)**
  - Reverses the truth/falsity of its condition
  - Returns **`true`** when its condition is **`false`**, & vice versa
  - Is a unary operator, only takes one condition
  - **Example**

```
if ( !( grade == sentinelValue ) )
   cout << "The next grade is " << grade << endl;
```

Alternative

```
if ( grade != sentinelValue )
   cout << "The next grade is " << grade << endl;
```

Logical Operators

- ! negates any boolean expression
  - !(x < y)
    - True if x is NOT less than y
  - !(x = = y)
    - True if x is NOT equal to y
- ! Operator can make expressions difficult to understand…use only when appropriate

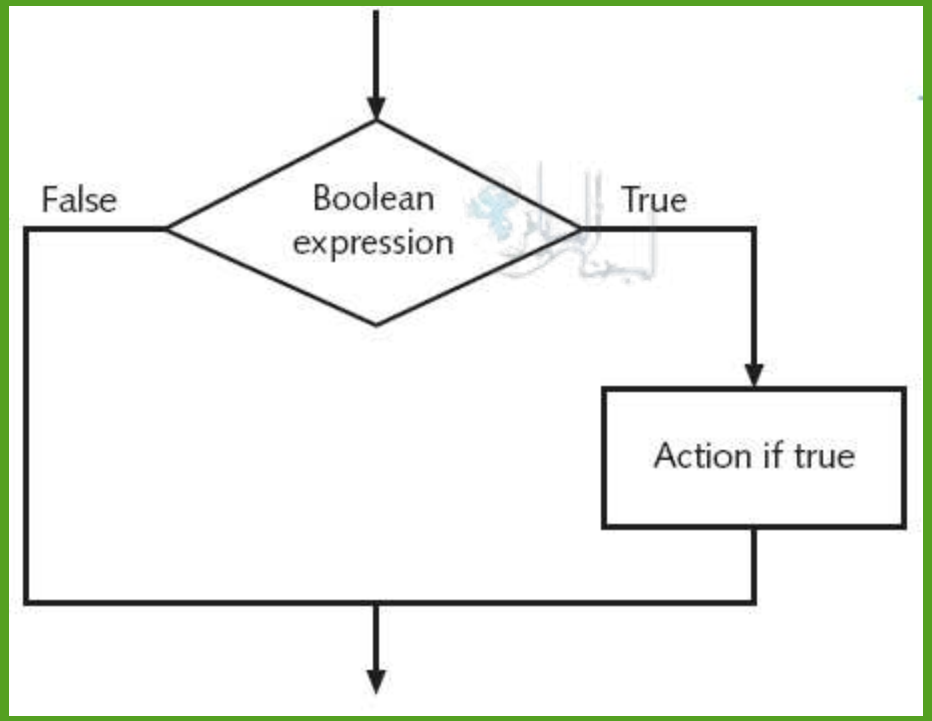
Truth table for the ! (NOT, logical negation) operator

expression	!expression
false	true
true	false

Operator	Description	Associativity
()	Grouping operator	left to right
! ++ -- - +	Logical negation Pre-increment Pre-decrement Unary minus Unary plus	right to left
* / %	Multiplication Division Modulus	left to right
+ -	Addition Subtraction	left to right
< <= > >=	Comparison less-than Comparison less-than-or-equal-to Comparison greater-than Comparison geater-than-or-equal-to	left to right
= = !=	Comparison equal-to Comparison not-equal-to	left to right
&&	Logical AND	left to right
||	Logical OR	left to right
? :	Ternary conditional (if-then-else)	right to left
= += -= *= /= %=	Assignment operator Increment and assign Decrement and assign Multiply and assign Divide and assign Modulo and assign	right to left

Control structures

If selection structure flow chart



If selection structure

- example:

Pseudocode

If student's grade is greater than or equal to 60

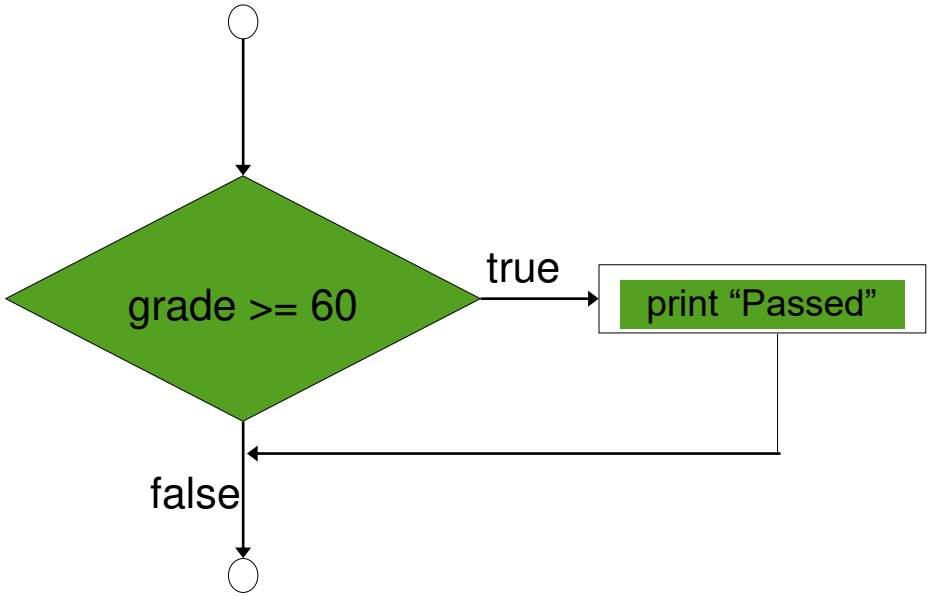
Print "Passed"

C++ code

```
if ( grade >= 60 )
   cout << "Passed";
```

If selection structure

- Flow chart for the pseudocode statement.



A decision can be made on any expression.

If/else selection structure

- *Different actions if conditions true or false*

- **Syntax**

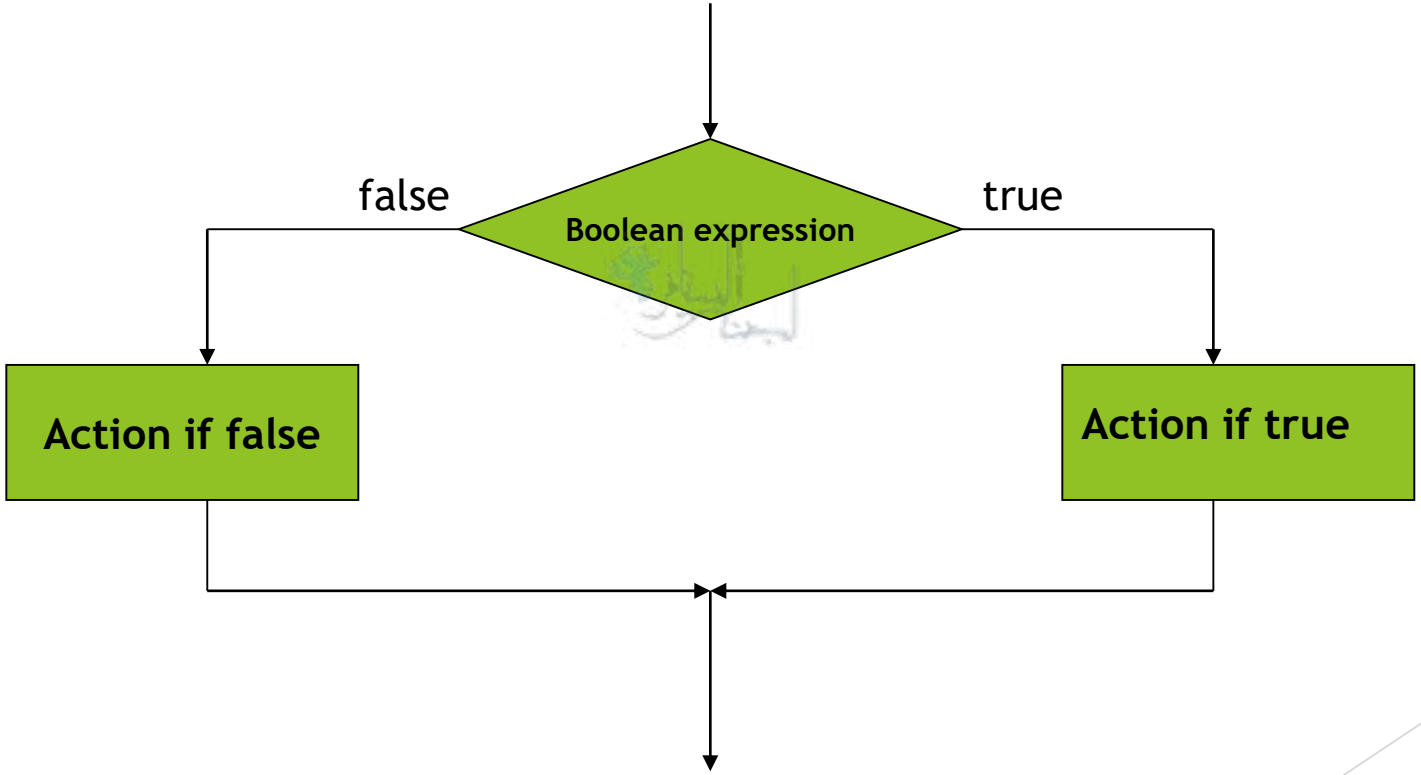
- A single statement for each alternative

```
if  (Boolean_expression)
    yes_statement;
else
    no_statement;
```

- A sequence statement for each alternative

```
if  (Boolean_expression)
    {   yes_statement1;
        yes_statement2;
        yes_statement last; }
else
    {   no_statement1;
        no_statement2;
        no_statement last; }
```

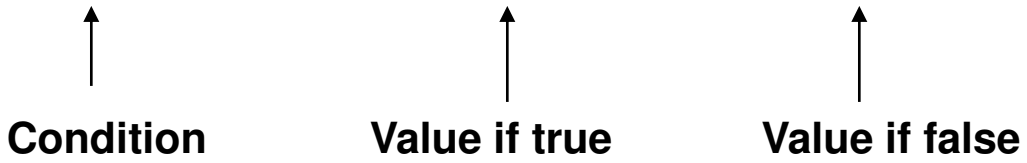
If/else selection structure flow chart



If/else selection structure

- **Ternary conditional operator (?:)**
- It is only ternary operator.
- It takes three operands together with conditional operator to form a conditional expression
  - Three arguments (condition, value if `true`, value if `false`)
  - example

```
cout << ( grade >= 60 ? "Passed" : "Failed" );
```



- the value in the conditional can also be actions to execute for example

```
grade >= 60 ? cout << "Passed" : cout << "Failed" ;
```

Nested if/else structure

- if structure that rests entirely within another if structure, within either the if or the else clause
  - One inside another, test for multiple cases
  - Once condition met, other statements skipped

Nested if/else structure

- following pseudocode is example of nested if/else
- *if student's grade is greater than or equal to 90*
  Print "A"
- *else*
  if student's grade is greater than or equal to 80
  Print "B"
  else
  if student's grade is greater than or equal to 70
  Print "C"
  else
  if student's grade is greater than or equal to 60
  Print "D"
  else
  - *Print "F"*

Nested if/else structure

C++ code

```
if ( grade >= 90 )        // 90 and above
       cout << "A";
    else

      if ( grade >= 80 )  // 80-89
          cout << "B";
     else

        if ( grade >= 70 )  // 70-79
            cout << "C";
      else

       if ( grade >= 60 )  // 60-69
           cout << "D";
        else                        // less than 60
           cout << "F";
```

- if grade>=90, first four conditions will be true. But only the **cout** statement after the first test will be executed. After that **cout** is executed, the **else** part of the outer **if/else** statement is skipped.

The switch Multiple-Selection Structure

- Switch is typically a selection structure and an alternative to the long string of **ifs**
- Tests variables for multiple values
- Consists of a series of **case** labels and an optional **default** case
- **General format**

```
switch ( variable ) {
 case value1:        // taken if variable == value1
 statements
 break;                  // necessary to exit switch

 case value2:
 case value3:     // taken if variable == value2 or == value3
 statements
 break;

 default:         // taken if variable matches no other cases
 statements
  break;
}
```

The switch Multiple-Selection Structure

- **Expression (variable in braces after switch)** must be of type integer or character. It is the controlling expression of switch statement

- The keyword **case** must be followed by a **constant**

- **break** statement is required unless you want all subsequent statements to be executed.

Repetition structures

Repetition structures

while Repetition Structure

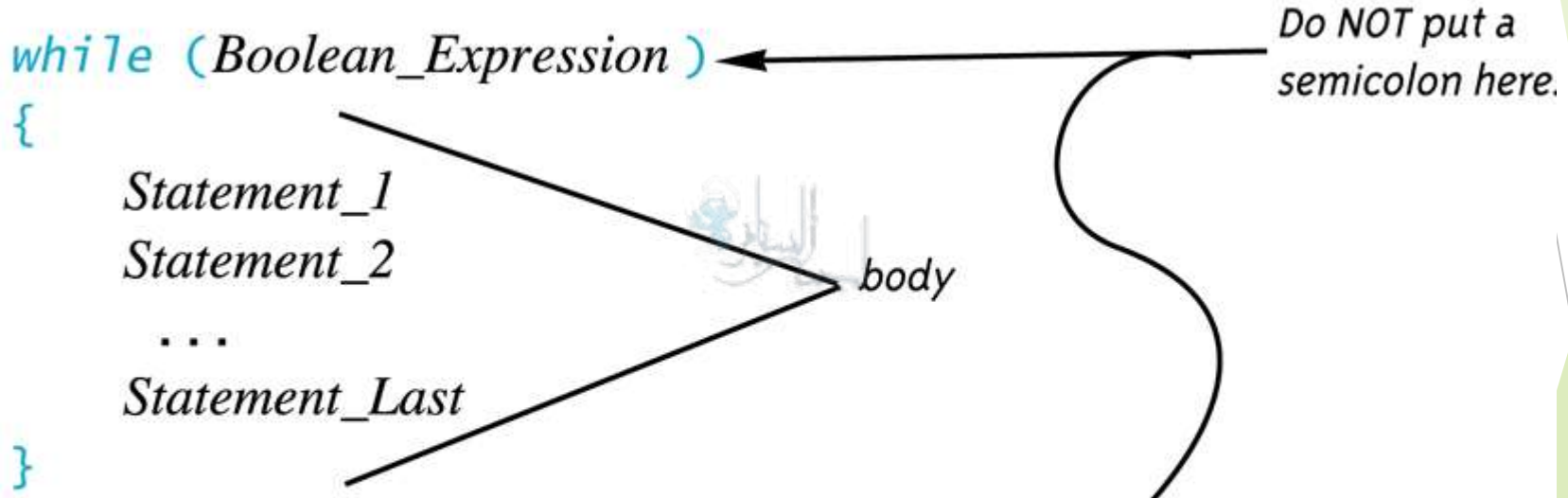
- Action repeated while some condition remains true.

While loop syntax

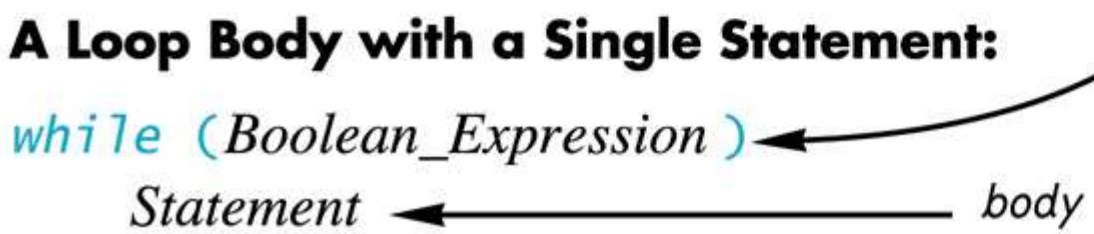
- while (*boolean expression is true)*
  {
  statements to repeat ; *//braces are required for compound statement.*
  }
- While (*boolean expression is true*)
  statement to repeat; *// no braces are required for single statement*

While statement syntax

A Loop Body with Several Statements:

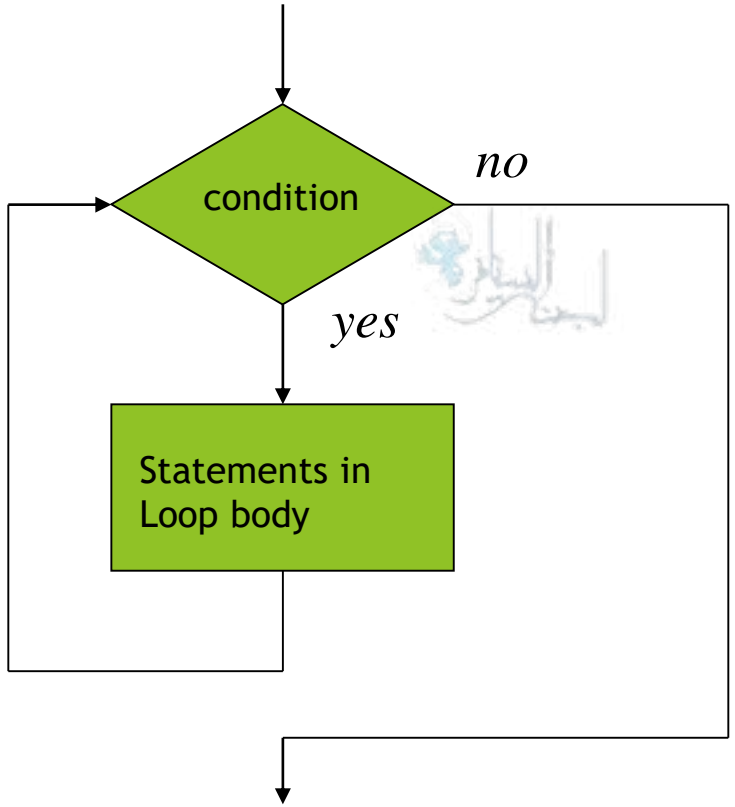


A Loop Body with a Single Statement:



While repetition structure

- **Flow chart**



for Repetition Structure

- **for** loop is used when the number of times to be repeated is fixed/known
- It handles all the details of the counter controlled repetition
- Execution continues as long as the boolean expression is true

- General format of the **for** loops

```
for (initialization; Loop Continuation Test; update)
{
statement block;
 }
```

  - **Initialization action**
    - Done only once at the start
    - Initialization expression initialize and declare the loop's control variable e.g. int counter = 1;
    - Notice that there is a semi colon after initialization statement

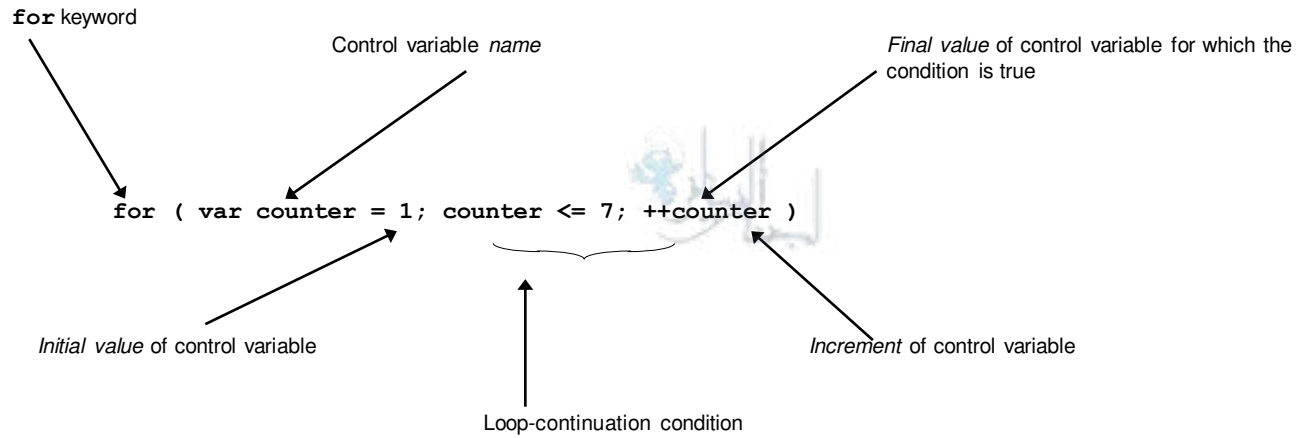
for Repetition Structure

- **Loop continuation test**
  - Is a boolean expression
  - Expreassion contains the final value of the control variable for which the condition is true e.g. counter <= 10;
  - Loop continuation test evaluates to true or false
  - If true body of **for** is executed, else exit **for** loop
  - Notice that there is also a semi colon after loop condition test
- **Update**
  - Specifies how to update condition
  - After the execution of the body of the loop, the condition of the loop is updated e.g. counter ++
  - Notice that there is no semi colon after updat_action
- **Example**

```
for( int counter = 1; counter <= 10; counter++ )
        cout << counter << endl;
```

  - Prints integers from one to ten

Components of typical for header



Examples Using the for Structure

- Vary control variable from 1 to 100 in increments of 1
  - `for (int i = 1, i <= 100; i++)`
- Vary control variable from 100 to 1 in decrements of -1
  - `for (int i = 100, i >= 1; i--)`
- Vary control variable from 7 to 77 in steps of 7
  - `for (int i = 7, i <= 77; i += 7)`
- Vary control variable from 20 to 2 in steps of -2
  - `for (int i = 20, i >= 2; i -= 2)`

Using multiple variables in for loop

- There may be several variables in the for structure that must be initialized and updated (e.g. incremented or decrement)

- Coma operator is used to enable the programmer to use multiple initialization and increment expressions

  - For multiple variables, use comma-separated lists

Example

```
for (int i = 0, j = 0;  j + i <= 10; j++, i++)
   cout << j + i << endl;
```

do-while Repetition Structure

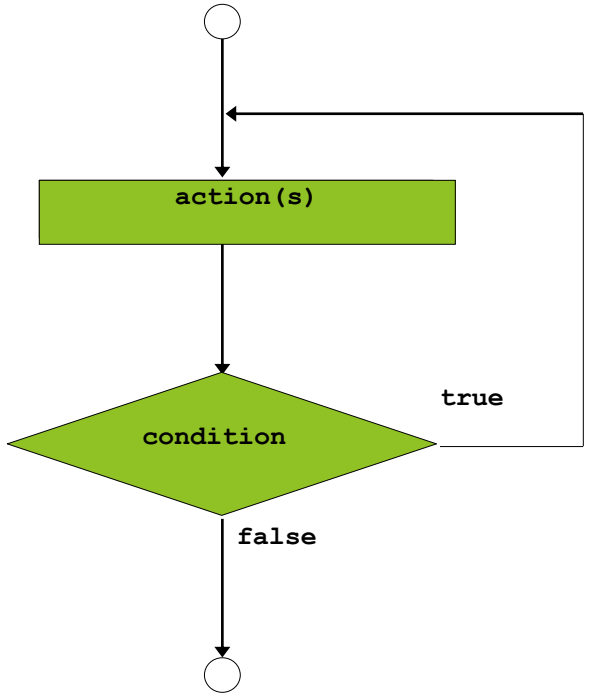
- A variation of the while loop.
- A do-while loop is always executed at least once
- Makes loop continuation test at the end not the beginning
  - The body of the loop is first executed
  - The condition (boolean expression) is checked after the body has been executed
- **Syntax / Format**

```
do {
    statement block
} while ( condition );
```

- **semantics**

1. Execute the statement.
2. Evaluate the expression.
3. If it is TRUE then proceed to step (1) else exit the loop

do-while flow chart




```
do
    statement;
while (condition);
```

Or

```
do
{
    statement block
} while (condition);
```

Break and continue Statements

- The break and continue statements alter the flow of control

- **break statement**
  - Causes immediate exit from **`while`**, **`for`**, **`do/while`** or **`switch structure`**
  - Program execution continues with first statement after structure
- **Common uses**
  - Escape early from a loop
  - Skip the remainder of **`switch structure`**

Break and continue Statements

- **continue statement**
  - Used in **`while`**, **`for`**, **`do/while`**
  - Skips remainder of loop body
  - Proceeds with next iteration of loop
- **`while`** and **`do/while`** structure
  - Loop-continuation test evaluated immediately after the **`continue`** statement
- **`for`** structure
  - Increment expression executed
  - Next, loop-continuation test evaluated

LECTURE 12
Arrays

Array

- **Arrays** are data structures consisting of related data items of the same type
- An **array** is used to process a collection of data of the same type
  - Examples: A list of names
    A list of temperatures
- Arrays
  - are static entities (same size throughout program)
  - Consecutive group of memory locations
  - Same name and type (`int`, `char`, etc.)
- Arrays can be of type
  - Pointer based arrays (C-style) used in this chapter
  - Arrays as objects (C++)

The Array Variables

- The variables making up the array are referred to as
  - **Indexed variables**
  - **Subscripted variables**
  - **Elements of the array**

- The number of indexed variables in an array is the **declared size**, or **size**, of the array
  - The largest index is one less than the size
  - The first index value is zero

Declaring an Array

- When declaring arrays, programmer specify
  - Name
  - Type of array
    - Any data type
  - Number of elements (array size)
- Format
  - `type arrayName[ arraySize ];`
- Example

```
int c[ 10 ];  // array of 10 integers
float d[ 3284 ]; // array of 3284 floats
```

- Declaring multiple arrays of same type
  - Use comma separated list, like regular variables

```
int b[ 100 ], x[ 27 ];
```

Array Variable Types

- An array can have indexed variables of any type
- All indexed variables in an array are of the same type
  - This is the **base type** of the array
- An indexed variable can be used anywhere an ordinary variable of the base type is used

Using [ ] With Arrays

- In an array declaration, [ ]'s enclose the size of the array such as this array of 5 integers:

  int score [5];

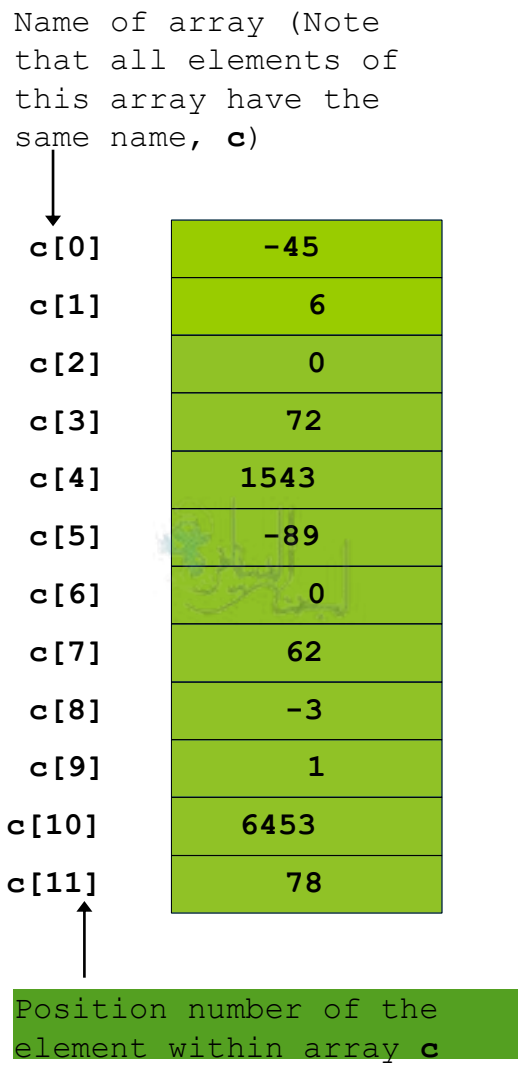
- When referring to one of the indexed variables, the [ ]'s enclose a number identifying one of the indexed variables
  - `score[3]` is one of the indexed variables
  - The value in the [ ]'s can be any expression that evaluates to one of the integers 0 to (size -1)

Array

- To refer to a particular location or an element in array
  - Specify array name and position number (index)
  - Format
    - `arrayname[ position number ]`
  - First element in every array is ***zeroth element***
- Consider N-element array c
  - `The elements are c[ 0 ], c[ 1 ] … c[ n - 1 ]`
  - Nth element as position N-1
- Subscript
  - The position number within square brackets
  - Must be an integer o an integer expression (any integral type)

Array

A 12-element Array named **c**



Array

- Array elements like other variables
  - Assignment, printing for an integer array **c**

```
c[ 0 ] =  3;
cout << c[ 0 ];
```

  - Performing division on array element

```
x = c[ 6 ] / 2; /*divide seventh element of array
by 2 and assign the result to the variable x */
```

  - printing sum of first three array elements of array **c**

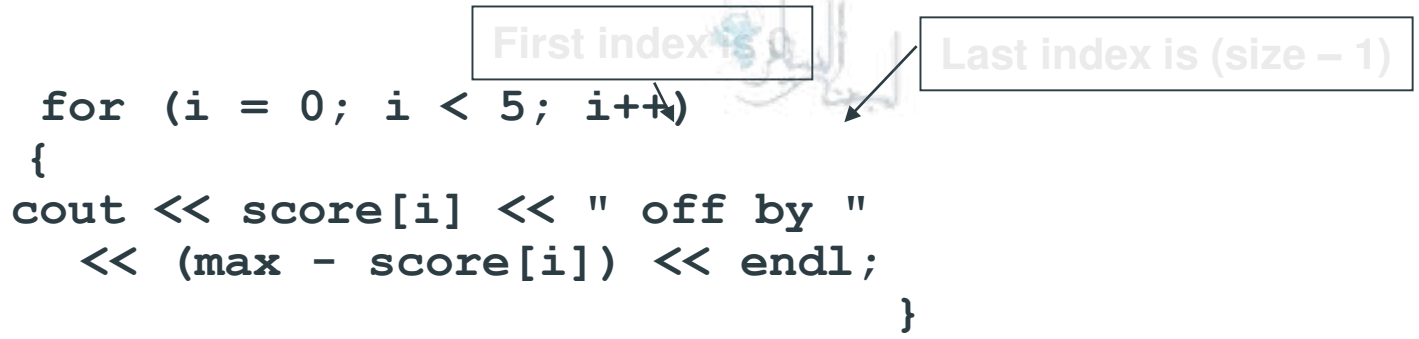
```
cout << c[ 0 ] + c[ 1 ] + c[ 2 ];
```

- Can perform operations inside subscript

```
c[ 5 - 2 ]
```
same as `c[3]`

Loops And Arrays

- **For loop**
  - for-loops are commonly used to step through arrays
  - Set each element
  - Example



  - could display the difference between each score and the maximum score stored in an array

Initializing Arrays

- To initialize an array when it is declared
  - The values for the indexed variables are enclosed in braces and separated by commas
- Example

```
int children[3] = { 2,  12,  1 };
```

- Is equivalent to:

```
int children[3];
        children[0] = 2;
        children[1] = 12;
    children[2] = 1;
```

Initializing Arrays

- If too few values are listed in an initialization statement
  - The listed values are used to initialize the first of the indexed variables
  - The remaining indexed variables are initialized to a zero of the base type
  - Example: `int a[10] = {5, 5};`
    initializes `a[0]` and `a[1]` to `5` and `a[2]` through `a[9]` to `0`
- If values listed in initialization statement are more than array elements, it causes a syntax error

Initializing Arrays

- To set every element to same value

```
int n[ 5 ] = { 0 };
```

- If array size omitted, values listed in initialization statement determine size

```
int n[] = { 1, 2, 3, 4, 5 };
```

  - 5 initializers, therefore 5 element array

Array size (constant variables)

- Array size Can be specified with constant variable (**const**)
  - **const int size = 20;**
- Also called named constants or read-only variables
- Use constants to declare the size of an array
  - Using a constant allows your code to be easily altered for use on a smaller or larger set of data

  - **Example**

```
//Constants must be initialized when declared and can not be modified
  there after

  const int  NUMBER_OF_STUDENTS = 50;
  int score[NUMBER_OF_STUDENTS];
  for ( i = 0; i < NUMBER_OF_STUDENTS; i++)

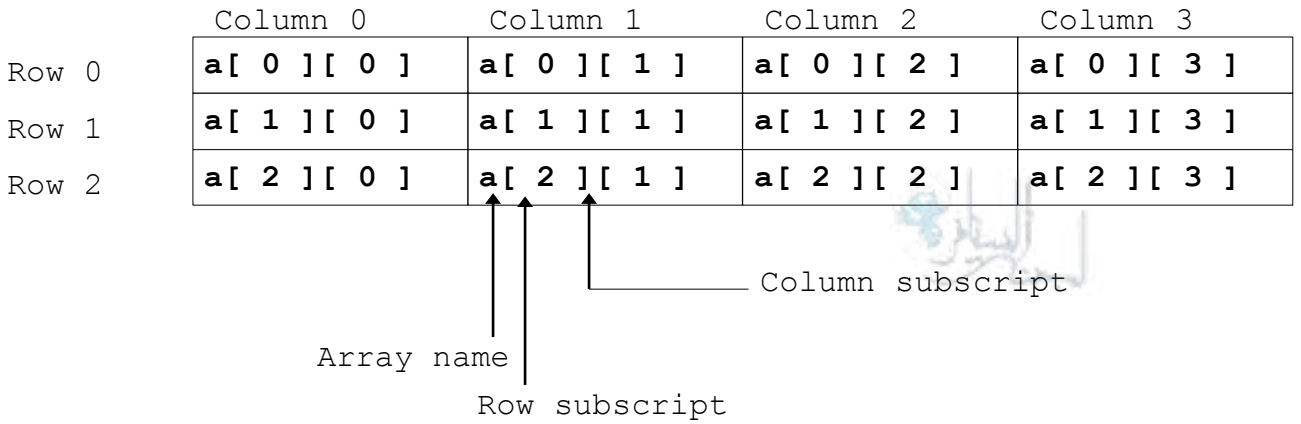
    cout << score[i] << " off by "<< (max - score[i]) << endl;
```

  - Only the value of the constant can be changed to make this code work for any number of students

Two dimensional Arrays

- **Two Dimensional Arrays**
  - To identify a particular table element two subscripts must be specified
  - `a[ i ][ j ]`
    - **a** is the name of array
    - **i** and **j** are the subscripts that uniquely identify each element
  - Specify row, then column
    - `(subscripts [i]=row & [j]=column)`

- A double subscript array with 3 rows and 4 columns



Initializing Multiple-Subscripted Arrays

- A multiple subscript array can be initialized in its declaration much like a single subscripted array
- **To initialize**
  - Initializers grouped by row in braces

```
int b[ 2 ][ 2 ] = { { 1, 2 }, { 3, 4 } };
```

    - `1` and `2` initialize `b[0][0]` and `b[0][1]`
    - `3` and `4` initialize `b[1][0]` and `b[1][1]`

1	2
3	4

  - Default of 0
    - If not enough initializers for a given row, remaining elements are initialized to zero

```
int b[ 2 ][ 2 ] = { { 1 }, { 3, 4 } };
```

1	0
3	4

Multiple-Subscripted Arrays

- Referenced like normal

1	0
3	4

```
cout << b[ 0 ][ 1 ];
```

  - Outputs 0
    - 0 is element `b[0][1]` of `array b`

  - Cannot reference using commas

```
cout << b[ 0, 1 ];
```

    - Syntax error

Functions

Modular programming with Functions

- C++ programs can be written by combining
  - pre-packaged functions
  - Programmer defined functions

- **Pre- packaged functions**
  - Are available in standard C++ library
  - Provided as part of the C++ programming environment
  - C++ library provides rich collection of functions for performing
    - Common mathematical calculations
    - String manipulations
    - Character manipulations
    - Input/output, error checking and many other useful operations

Modular programming with Functions

- **Programmer defined functions**
  - Programmer can write functions to define specific tasks
  - Could be used at many points in a program
  - Actual statements defining the function are written only once
    - These statements are hidden from other functions

- **Calling a function**
- Function is invoked by function call
- A function call specifies the function name and provides information (as arguments) that the called function needs
- Functions called by writing
  - **functionName (argument);**

  or

  - **functionName(argument1, argument2, …);**

Function Libraries

- Predefined functions are found in libraries

- The library must be “included” in a program to make the functions available

- An include directive tells the compiler which library header file to include.

- To include the math library containing sqrt():

```
#include <cmath>
```

- Newer standard libraries, such as cmath, also require the directive

```
using namespace std;
```

Modular programming with Functions

- **Math library functions**
  - Allow the programmer to perform common mathematical calculations
  - To use math library functions including the header file **<cmath>**

- **Example**

```
cout << sqrt( 900.0 );
```

  - Calls the **sqrt** (square root) function.
  - The preceding statement would print **30**
  - The **sqrt** function takes an argument of type **double**
  - And returns a result of type **double**, as do all functions in the math library

Modular programming with Functions

- Function call arguments can be
  - Constants

```
sqrt( 4 );
```

  - Variables

```
sqrt( x );
```

  - Expressions

```
sqrt( sqrt( x ) );
sqrt( 3y + 6 );
```

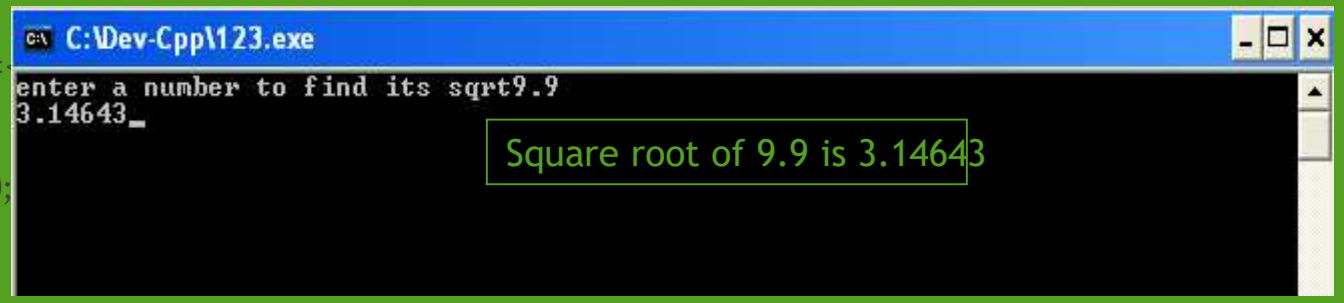
Commonly used math library functions

Method	Description	Example
ceil(x)	rounds *x* to the smallest integer not less than *x*	ceil(9.2) is 10.0 ceil(-9.8) is -9.0
cos(x)	trigonometric cosine of *x* (*x* in radians)	cos(0.0) is 1.0
exp(x)	exponential function *ex*	exp(1.0) is 2.71828 exp(2.0) is 7.38906
fabs(x)	absolute value of *x*	fabs(5.1) is 5.1 fabs(0.0) is 0.0 fabs(-8.76) is 8.76
floor(x)	rounds *x* to the largest integer not greater than *x*	floor(9.2) is 9.0 floor(-9.8) is -10.0
fmod(x, y)	remainder of *x*/*y* as a floating-point number	fmod(13.657, 2.333) is 1.992
log(x)	natural logarithm of *x* (base *e*)	log(2.718282) is 1.0 log(7.389056) is 2.0
log10(x)	logarithm of *x* (base 10)	log10(10.0) is 1.0 log10(100.0) is 2.0
pow(x, y)	*x* raised to power *y* (*xy*)	pow(2, 7) is 128 pow(9, .5) is 3
sin(x)	trigonometric sine of *x* (*x* in radians)	sin(0.0) is 0
sqrt(x)	square root of *x*	sqrt(900.0) is 30.0 sqrt(9.0) is 3.0
tan(x)	trigonometric tangent of *x* (*x* in radians)	tan(0.0) is 0

Program example to observe predefined functions

```
1.  //program to find the square root of any number
2.  //by calling a predefined math function for square root
3.  #include <iostream>
4.  #include <cmath>    //math library header file
5.  using namespace std;
6.  int main()
7.  {
8.      double x;
9.      cout<<"enter a number to find its sqrt";
10.      cin>>x;

11.     cout<<
12.
13. return 0;
14. }
```



Fundamentals of Characters and Strings

Fundamentals of Characters and Strings

- **Character constant**
  - Integer value represented as character in single quotes
  - `'z'` is integer value of `z`
    - `122` in ASCII character set
  - `'\n'` is integer value of **`newline`**
    - `10` in ASCII character set

Fundamentals of Characters and Strings

- **String**
  - Series of characters treated as a single unit
  - Can include letters, digits, special characters `+, -, * ...`
  - String literal (string constants)
    - Enclosed in double quotes, for example:

```
"I like C++"
```

  - String is an Array of characters, ends with null character `'\0'`
  - String is a constant pointer
    - Pointer to string's first character
      - Like arrays

Fundamentals of Characters and Strings

- **String assignment**
  - A string can be assigned in declaration to character array or variable of type **`char *`**
  - **Character array**
    - **`char color[] = "blue";`**
      - Creates 5 element **`char`** array **`color`**
        - last element is **`'\0'`**
  - **Alternative for character array**
    - **`char color[] = { 'b', 'l', 'u', 'e', '\0' };`**

String Manipulation Functions of the String-handling Library <string.h>

Function prototype	Function description
`char *strcpy( char *s1, const char *s2 );`	Copies the string **s2** into the character array **s1**. The value of **s1** is returned.
`char *strncpy( char *s1, const char *s2, size_t n );`	Copies at most **n** characters of the string **s2** into the character array **s1**. The value of **s1** is returned.
`char *strcat( char *s1, const char *s2 );`	Appends the string **s2** to the string **s1**. The first character of **s2** overwrites the terminating null character of **s1**. The value of **s1** is returned.
`int strcmp( const char *s1, const char *s2 );`	Compares the string **s1** with the string **s2**. The function returns a value of zero, less than zero or greater than zero if **s1** is equal to, less than or greater than **s2**, respectively.
`size_t strlen( const char *s );`	Determines the length of string **s**. The number of characters preceding the terminating null character is returned.

Functions

- **Functions**
  - Modularize a program
  - Software reusability
    - Call function multiple times
- **Local variables**
  - Known only in the function in which they are defined
  - All variables declared in function definitions are local variables
- **Parameters**
  - Local variables passed to function when called
  - Provide outside information

Function call

- Function is invoked by function call
- A function call specifies the function name and provides information (as arguments) that the called function needs
- Functions called by writing
  - **functionName (argument);**

  or

  - **functionName(argument1, argument2, …);**

- The argument can be a constant, variable or expression

Function Definitions

- Each program we observed so far consisted of a function called **main**
- Function main called standard library functions to accomplish its task
- Now we are going to discuss how programmer write there own customized functions
- For programmer defined functions the important components are
  - Function definition
  - Function prototype

Function Definition

- Describes how the function does its task
- Can appear before or after the function is called
- **Format**

```
return-value-type  function-name ( parameter-list )
    {
      declarations and statements
    }
```

  - **Function-name**
    - is any valid identifier e.g. square
  - **Return-value-type**
    - is the data type of the result returned from the function to the caller
    - Return value type **void** indicated that the function does not return a vlaue
  - **Parameter-list**
    - Is a comma-separated list of the arguments
    - containing the declarations (data type needed for each argument) of the parameters received by the function when it is called
    - If the function does not receive any values, **parameter-list** is **void** or simply left **empty**

Function Definition

- Example function

```
int square( int y )
{
  return y * y;
}
```

- `return` keyword
  - **Format** ***return expression;***
  - Returns the value calculated by the function
  - Returns data, and control goes to function's caller
    - If no data to return, use `return;`
  - Function ends when reaches right brace
    - Control goes to caller
- *Functions cannot be defined inside other functions*

Function Prototype

- Must appear in the code before the function can be called
- The compiler use the function prototypes to validate function call
- **Format /syntax**
  - ***Return-value-type Function_Name(Parameter_List);***

- Function prototype tells the compiler
  - **The function's name**
  - **Type of data returned by the function** (void if return nothing)
  - **Number of parameters the function accepts to receive**
  - **Types of parameters**
  - **The order in which these parameters are expected**
- Function prototype is not required if the function definition appears before the first use function's first use in the program.
- In such case the function definition also acts as the function prototype

Example program

```
1  // program calculate the square of integers 1 to 10
2  // Creating and using a programmer-defined function.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  int square( int );  // function prototype
9
10 int main()
11 {
12    // loop 10 times and calculate and output
13    // square of x each time
14    for ( int x = 1; x <= 10; x++ )
15       cout << square( x ) << " ";  // function call
16
17    cout << endl;
18
19    return 0;  // indicates successful termination
20
21 } // end main
23 // square function definition returns square of an integer
24 int square( int y )  // y is a copy of argument to function
25 {
26    return y * y;    // returns square of y as an int
27
28 } // end function square
```

Function prototype: specifies data types of arguments and return values. **`square`** expects and **`int`**, and returns an **`int`**

Parentheses **`()`** cause function to be called. When done, it returns the result.

Definition of **`square`**. **`y`** is a copy of the argument passed. Returns **`y * y`**, or **`y`** squared.

Output



Function prototype

- Prototype must match function definition

  - Function prototype

```
double maximum( double, double, double );
```

  - Definition

```
double maximum( double x, double y, double z )
{
  …
}
```

- observe this in the following program example
- This example program uses a programmer-defined function maximum to determine and return the largest of three integers

Example program

```
1   // using programmer defined function
2   // Finding the maximum of three floating-point numbers.
3   #include <iostream>
4
5   using std::cout;
6   using std::cin;
7   using std::endl;
8
9   double maximum( double, double, double ); // function prototype
10
11  int main()
12  {
13     double number1;
14     double number2;
15     double number3;
16
17     cout << "Enter three floating-point numbers: ";
18     cin >> number1 >> number2 >> number3;
19
20     // number1, number2 and number3 are arguments to
21     // the maximum function call
22     cout << "Maximum is: "
23          << maximum( number1, number2, number3 ) << endl;
24
25     return 0;  // indicates successful termination
```

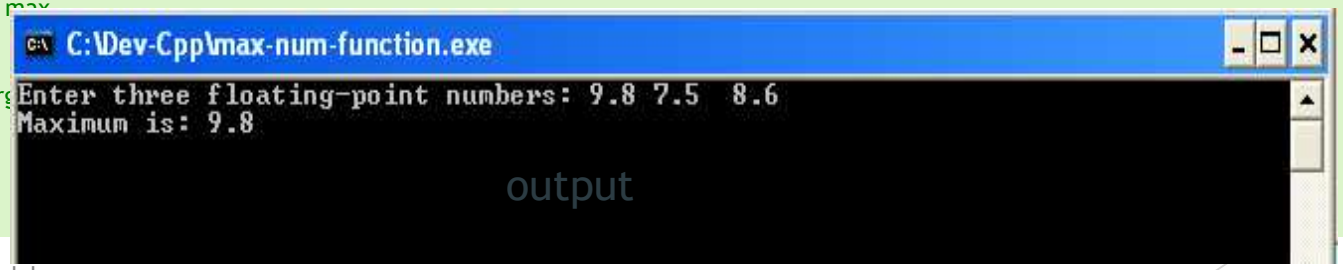


Function **`maximum`** takes 3 arguments (all **`double`**) and returns a **`double`**.

Example program

Comma separated list for multiple parameters.

```
26
27  } // end main
28
29  // function maximum definition;
30  // x, y and z are parameters
31  double maximum( double x, double y, double z )
32  {
33     double max = x;   // assume x is largest
34
35     if ( y > max )    // if y is larger,
36        max = y;       // assign y to max
37
38     if ( z > max )    // if z is larger,
39        max = z;       // assign z to max
40
41     return max;       // max is larg
42
43  } // end function maximum
```



output

LECTURE 9

Local Variables

- Variables declared in a function:
  - Are **local** to that function, they cannot be used from outside the function
  - Have the function as their **scope**

- Variables declared in the main part of a program:
  - Are local to the main part of the program, they cannot be used from outside the main part
  - Have the main part as their scope

Global Variables

- Global Variable --
- used when more than one function must use a common variable

Static storage class

- `static` keyword
  - Local variables in function declared with keyword **static**
  - Keeps value between function calls
  - Only known in own function
  - Static local variables retain their values when function is exited
  - `static int count = 1; /* declare local variable count to be static and initialized to 1`
  - Local variable Initialized to **0** by default

Scope Rules

- **Scope**
  - Portion of program where identifier can be used
- Scopes for an identifier are
  - Function scope
  - File scope
  - Block scope
  - Function prototype scope

Scope Rules

- Function scope
  - Can only be referenced inside function in which they appear, can not be referenced outside function body
  - Only labels are the identifiers with function scope
    - labels are identifiers with a colon
    - e.g. `case:` (as used in switch)

Scope Rules

- Block scope
  - Identifiers declared inside the block
  - Begins at declaration, ends at right brace } of the block
    - Can only be referenced in this range
  - Local variables, function parameters
  - In case of nested blocks and identifiers in inner and outer block have same name
    - Identifier in the outer block is hidden until the inner block terminates
  - `static` variables still have block scope
    - Storage duration does not effect the scope of block

Scope Rules

- Function-prototype scope
  - Only identifiers with this scope are those used in Parameter list of prototype
  - Names in function prototype optional, only type required
    - Compiler ignores the name if used in parameter list of function-prototype
  - In a single prototype, name can be used once
  - Identifier used in function prototype can be reused elsewhere in the program

Passing Arrays to functions

- To pass an array to a function
  - Specify the name of Array without brackets
  - **For Example**

```
int myArray[ 24 ];
myFunction( myArray, 24 );
```

    - Pass array **`myArray`** to function **`myFunction`**
    - Array size is usually passed
      - Useful to iterate over all elements

- C++ automatically passes arrays to functions using simulated call-by-reference

Passing Arrays to functions

- Arrays are passed by call-by-reference
  - Functions can modify original array data
  - Value of name of array is address of first element of array
    - Function knows where the array is stored
    - Can modify the actual array elements in their original memory locations
- Individual array elements passed by call-by-value
  - Like regular simple variables
  - Such simple pieces of data are called scalar or scalar quantities
  - `square( myArray[3] );`

Passing Arrays to functions

- Indexed variables can be arguments to functions
  - **Example**

If a program contains these declarations:

```
int i, n, a[10];
void my_function(int n);
```

Variables **a[0]** through **a[9]** are of type int, making these calls legal

```
my_function( a[ 0 ] )
my_function( a[ 3 ] );
my_function( a[ i ] );
```

Passing Arrays to functions

- **Functions taking arrays**
  - For a function receiving array through function call, the function's parameter list must specify that an array will be received
  - Function prototype
    - `void modifyArray( int b[], int arraySize );`
    - `void modifyArray( int [], int );`

    `this prototype can also be written as`

    `void modifyArray(int anyArrayName[], int  anyVariableName);`
    - Names optional in prototype
    - Both take an integer array and a single integer

Passing Arrays to functions

- An array parameter is indicated using empty brackets in the parameter list
- **For example**

```
void fill_up(int a[ ], int size);
```

  - No need for array size between brackets
    - Ignored by compiler

Example of Function Calls With Arrays

- If function **`fill_up`** is declared in this way:

```
void fill_up(int a[ ], int size);
```

- and array **`score`** is declared this way:

```
int score[5], number_of_scores;
```

- **`fill_up`** is called in this way:

```
fill_up( score, number_of_scores );
```

Using const with Arrays

- If const is used to modify an array parameter
  - const is used in both the function declaration and definition to modify the array parameter
  - The compiler will issue an error if you write code
    that changes the values stored in the array parameter

Multiple-Subscripted Arrays

- Function prototypes
  - Must specify sizes of subscripts
    - First subscript not necessary, as with single-scripted arrays

```
void printArray( int [][ 3 ] );
```

    - All subsequent subscript sizes are required
    - The compiler uses the sizes to determine locations of elements in memory
    - Array elements are stored consecutively in memory regardless of number of subscripts.
    - In double subscript array, first row is stored in memory followed by the second row.